

**REPÚBLICA BOLIVARIANA DE VENEZUELA
MINISTERIO DEL PODER POPULAR PARA LA EDUCACIÓN SUPERIOR
UNIVERSIDAD NACIONAL EXPERIMENTAL
DE LAS TELECOMUNICACIONES E INFORMÁTICA**

Taller 2: Productos de Calidad

Miguel Figuera C.I: 23.558.789

Iromy León C.I: V-30.243.131

Alejandra Herde C.I: V-23.711.974

Tutor: Misley Baute

Caracas, Mayo de 2026

Tabla de Contenido

Introducción	3
Definiciones clave	4
Características de los requisitos	4
Diferencias entre ERS y DRS	5
Importancia del diseño en ingeniería de software	6
Conclusiones	11
Referencias	12

Introducción

El diseño de productos de software constituye una fase crucial dentro del ciclo de vida del desarrollo, especialmente en sistemas críticos como los de gestión veterinaria, donde la confiabilidad, disponibilidad y facilidad de uso impactan directamente en la atención animal. El presente taller aborda definiciones fundamentales, características de los requisitos, diferencias técnicas entre la Especificación de Requisitos de Software (ERS) y el Documento de Requisitos de Software/Diseño de Requisitos (DRS), la importancia del diseño y, como eje central, la arquitectura de PawCare, una aplicación moderna para veterinarias construida con Ruby on Rails 8, React 19 e Inertia.js bajo el patrón Model-React-Controller (MRC).

2. Definiciones: Especificación, ERC, norma, diseñar

Según la IEEE Std 610.12-1990 y Pressman (2014), se definen los siguientes términos:

Especificación: Documento técnico que describe de manera completa, precisa y verificable las propiedades, interfaces, comportamientos y restricciones de un sistema o componente. Su propósito es servir como contrato entre el cliente y el equipo de desarrollo.

ERC (Especificación de Requisitos del Cliente): También conocida como declaración de necesidades del usuario. Es un documento inicial, redactado en lenguaje natural, que expresa los objetivos, deseos y expectativas del cliente sin detallar aspectos técnicos. En el caso de PawCare, la ERC incluiría necesidades como “registro de pacientes (mascotas)”, “agendamiento de citas” y “envío de recordatorios automáticos”.

Norma: Conjunto de reglas, estándares o directrices reconocidas por una autoridad técnica (ej. IEEE, ISO) que establecen prácticas uniformes para actividades de ingeniería. En ingeniería de software, normas como IEEE 830 guían la elaboración de requisitos.

Diseñar: Proceso de concebir, definir y estructurar la arquitectura, componentes, interfaces y datos de un sistema de software para satisfacer los requisitos especificados, optimizando atributos de calidad como mantenibilidad, escalabilidad y eficiencia.

3. Características de los requisitos (ejemplos) Los requisitos de software deben cumplir con atributos de calidad para garantizar su utilidad. A continuación se ejemplifican los más importantes en el contexto de PawCare:

Característica	Significado	Ejemplo en PawCare
----------------	-------------	--------------------

Claridad	Redacción sin ambigüedades, fácil de entender para todos los stakeholders.	“El sistema mostrará un calendario semanal con las citas programadas, coloreando las confirmadas en verde y las pendientes en amarillo.”
Compleitud	Incluye todas las funcionalidades, restricciones y respuestas ante situaciones normales y excepcionales.	“Si una cita no es confirmada 24 horas antes, el sistema enviará un recordatorio automático por correo electrónico y SMS. Si no hay respuesta 2 horas antes, la cita se liberará automáticamente.”
Verificabilidad	Existe un método objetivo (prueba, inspección, demostración) para comprobar su cumplimiento.	“El tiempo de carga del historial clínico de una mascota no superará los 1.5 segundos con 10,000 registros almacenados.”
Consistencia	No hay conflictos internos ni con otros requisitos.	No puede exigirse “el botón de cancelar cita está siempre visible” y “el botón de cancelar se oculta si la cita ya comenzó” sin especificar condiciones.
Modificabilidad	Los requisitos pueden cambiarse fácilmente sin afectar la estructura general.	Uso de tablas de trazabilidad y módulos independientes (ej. módulo de citas separado del de inventario).

4. Diferencias técnicas entre ERS y DRS

Si bien ambos documentos gestionan requisitos, sus propósitos, audiencias y niveles de abstracción son distintos.

Aspecto	ERS (Especificación de Requisitos de Software)	DRS (Documento de Requisitos de Software / Diseño de Requisitos)
1. Enfoque principal	Qué debe hacer el sistema desde la perspectiva del usuario y el negocio.	Cómo se estructurará el sistema para cumplir esos requisitos; incluye decisiones de diseño.
2. Audiencia objetivo	Clientes, usuarios finales (veterinarios, recepcionistas), validadores, gerentes de proyecto.	Arquitectos de software, diseñadores, desarrolladores, equipos de pruebas técnicas.

3. Nivel de detalle técnico	Alto nivel, orientado a comportamiento y restricciones externas.	Bajo nivel, incluye módulos, subsistemas, interfaces, patrones de arquitectura y restricciones tecnológicas (ej. “uso de Rails 8 con Inertia.js y React 19”).
4. Contenido típico	Historias de usuario, casos de uso, requisitos funcionales (ej. “registrar una mascota”) y no funcionales (ej. “tiempo de respuesta < 2s”).	Diagrama de componentes (modelo MRC), descripción de capas (frontend React, backend Rails), decisiones tecnológicas (MySQL 8+, Solid Queue, WebSockets).

En la práctica, la ERS es la entrada para el DRS, y el DRS guía la implementación. La norma IEEE 830 se asocia más a la ERS, mientras que IEEE 1016 (Description of Software Architectural Design) se relaciona con el DRS.

5. Importancia del diseño en ingeniería de software

El diseño es una actividad crítica porque:

- **Traduce requisitos en una solución real:** Conecta el “qué” (requisitos) con el “cómo” (arquitectura).
- **Previene retrabajos costosos:** Detectar errores en diseño es hasta 10 veces más barato que en implementación o pruebas (Boehm, 1981).
- **Define la calidad estructural:** Atributos como escalabilidad, mantenibilidad, seguridad y rendimiento dependen directamente del diseño.
- **Facilita la comunicación técnica:** Mediante diagramas y patrones estandarizados (UML, C4 model, MRC), el equipo comparte una visión común.
- **Permite la reutilización y estandarización:** Un buen diseño promueve componentes desacoplados, reutilizables y alineados a normas de la organización.

6. Diseño del proyecto: PawCare – Aplicación móvil para gestión veterinaria

PawCare es una aplicación moderna de gestión veterinaria diseñada bajo el patrón **Model-React-Controller (MRC)** utilizando **Ruby on Rails 8**, **React 19** e **Inertia.js** como puente, eliminando la necesidad de una API REST tradicional. A continuación, se describe su arquitectura, pila tecnológica y estructura.

6.1 Arquitectura de alto nivel: Model-React-Controller (MRC)

PawCare implementa el patrón MRC donde Inertia.js actúa como puente entre el frontend (React) y el backend (Rails), permitiendo una aplicación de una sola página (SPA) con enrutamiento del lado del servidor. No existe una capa de API explícita: las peticiones HTTP son manejadas por funciones TypeScript dedicadas.

[React Components (Frontend)]



Inertia.js (Bridge)



Rails Controllers (Backend)



ActiveRecord Models



MySQL Database

Beneficio clave: Se obtiene la experiencia de una SPA (interfaz fluida sin recargas completas) con la simplicidad del enrutamiento y controladores del servidor, evitando la duplicación de lógica que suele ocurrir en arquitecturas separadas frontend/backend con API.

6.2 Pila tecnológica completa Backend

Dependencia	Versión	Propósito
Ruby	3.3+	Lenguaje de programación
Rails	8.0.2	Marco web
MySQL	8.0+	Base de datos relacional
Solid Queue	Integrado	Trabajos en segundo plano (respaldado por BD)
Solid Cache	Integrado	Almacén de caché (respaldado por BD)
Solid Cable	Integrado	WebSockets (respaldado por BD)
rails-inertia	3.12.1	Adaptador del servidor para patrón MRC
vite_rails	Última	Integración de empaquetado de activos
rspec-rails	8.0.2	Framework de pruebas backend
factory_bot_rails	6.5.1	Generación de datos de prueba
faker	3.5.2	Datos falsos para pruebas
shoulda-matchers	7.0.1	Matchers de RSpec
dotenv-rails	3.1.8	Variables de entorno

Frontend

Dependencia	Versión	Propósito
React	19.2.0	Librería de UI
TypeScript	5.9.3	Seguridad de tipos
Vite	5.4.21	Herramienta de compilación
@inertiajs/react	2.2.15	Adaptador del cliente para patrón MRC
Tailwind CSS	4.1.17	Framework de estilos
shadcn/ui	-	Sistema de componentes UI

Radix UI	-	Primitivos de accesibilidad
Vitest	30.2.0	Framework de pruebas frontend
@testing-library/react	16.3.0	Pruebas de componentes
ESLint	9.39.1	Linting de código
Prettier	Última	Formateo de código

6.3 Sistema de diseño

El proyecto incluye un sistema de diseño completo basado en shadcn/ui con más de 50 componentes accesibles:

- Tokens CSS centralizados (app/frontend/styles/tokens.css)
- Paleta de colores personalizable en formato HSL
- Componentes accesibles con Radix UI
- Soporte para modo claro/oscuro

app/

```

├── controllers/    # MRC: Controladores Rails
├── models/        # MRC: Modelos ActiveRecord
├── frontend/      # MRC: React Components
│   ├── components/  # Componentes reutilizables
│   ├── pages/      # Componentes de página Inertia
│   ├── hooks/      # Hooks personalizados de React
│   ├── types/      # Tipos TypeScript
│   ├── utils/      # Funciones auxiliares
├── entrypoints/   # Puntos de entrada Vite

```

6.5 Atributos de calidad considerados en el diseño

- **Rendimiento:** Al ser SPA con Inertia.js, las transiciones entre páginas son rápidas. El caché Solid Cache reduce consultas repetitivas a la base de datos.
- **Escalabilidad:** Separación clara de responsabilidades (controladores, modelos, componentes) permite escalar horizontalmente. Solid Queue gestiona trabajos asíncronos (ej. envío de recordatorios) sin bloquear la interfaz.
- **Mantenibilidad:** El uso de TypeScript y ESLint garantiza consistencia tipográfica y sintáctica. El patrón MRC, al eliminar una capa de API intermedia, reduce código boilerplate.
- **Disponibilidad:** WebSockets con Solid Cable permiten notificaciones en tiempo real (ej. “nueva cita asignada”).
- **Seguridad:** Autenticación gestionada por Rails (has_secure_password o Devise), protección CSRF integrada, y validaciones a nivel de modelo.
- **Accesibilidad:** Componentes Radix UI garantizan estándares WAI-ARIA.

Conclusiones

La correcta definición de especificación, requisitos del cliente, normas y diseño sienta las bases de cualquier proyecto de software exitoso, incluyendo sistemas verticales como el de gestión veterinaria.

Los requisitos deben poseer claridad, completitud, verificabilidad, consistencia y modificabilidad para evitar ambigüedades y fallos tempranos. En PawCare, estos atributos se verifican mediante pruebas automatizadas (RSpec + Vitest).

La ERS se enfoca en el comportamiento esperado desde la óptica del negocio veterinario (ej. “gestionar citas”), mientras que el DRS detalla la solución técnica (ej. “uso de Rails 8 con Inertia.js y patrón MRC”); confundirlos genera sobrecostos y entregas desalineadas.

El diseño es la etapa donde se definen la arquitectura y atributos de calidad; su importancia radica en prevenir retrabajos y garantizar la evolución del sistema.

PawCare demuestra un diseño moderno y eficiente: el patrón Model-React-Controller con Inertia.js elimina la complejidad de una API separada, ofreciendo una SPA con enrutamiento del servidor, respaldada por una pila tecnológica sólida (Rails 8, React 19, MySQL, Solid Queue) y un sistema de diseño accesible (shadcn/ui, Radix UI).

Referencias

Boehm, B. W. (1981). Software engineering economics. Prentice-Hall.

IEEE. (1990). *IEEE Standard Glossary of Software Engineering Terminology (IEEE Std 610.12-1990)*.

IEEE. (1998). *IEEE Recommended Practice for Software Requirements Specifications (IEEE Std 830-1998)*.

Pressman, R. S. (2014). Ingeniería del software: Un enfoque práctico (7ª ed.). McGraw-Hill.

Sommerville, I. (2011). Software engineering (9th ed.). Addison-Wesley.

PawCare Mobile

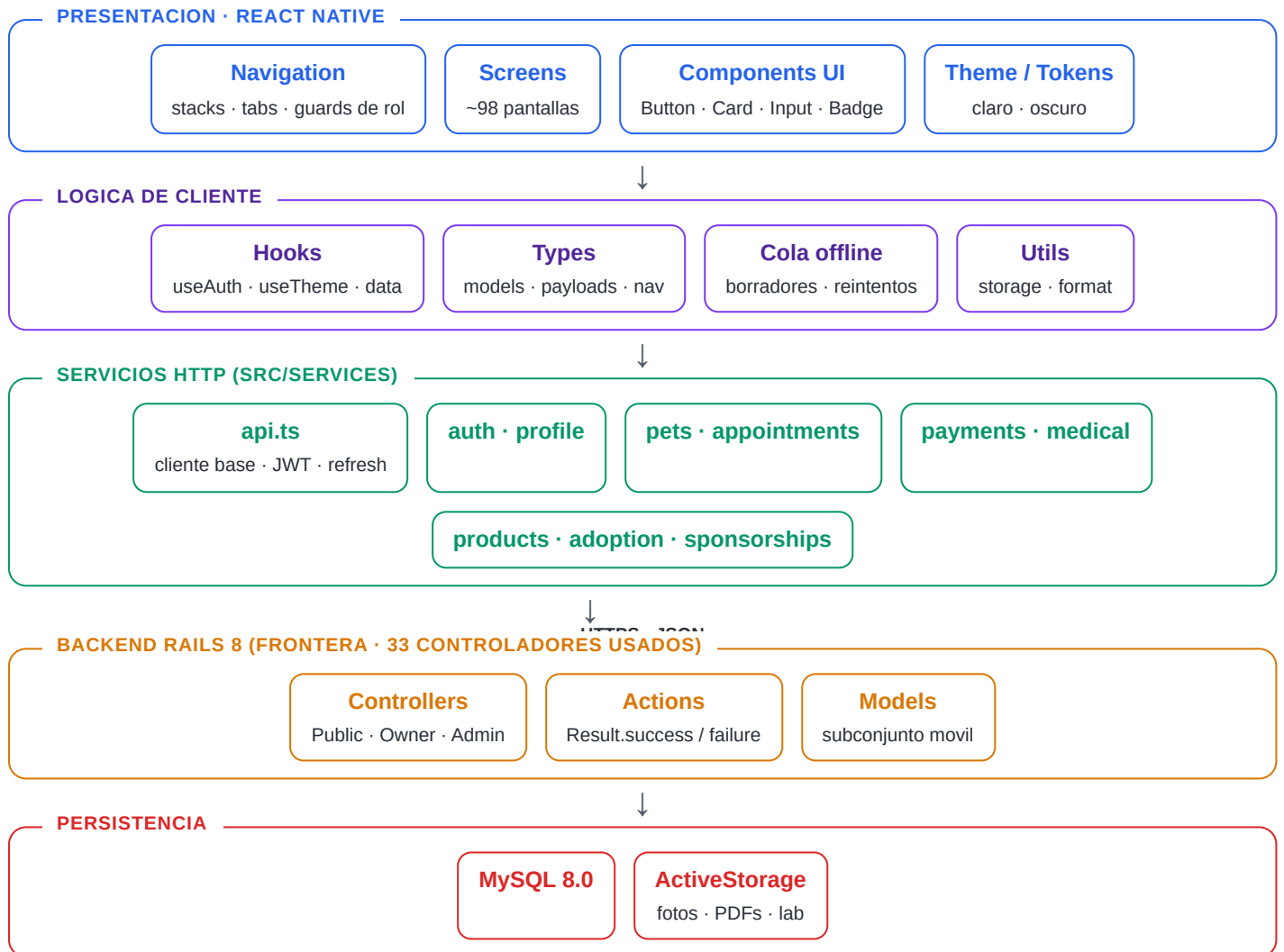
Diagramas de arquitectura — aplicacion movil (Expo SDK 56 · React Native · TypeScript)

Alcance: este documento cubre **unicamente lo que consume la app movil** — las **157 rutas** de `mobile-routes.md` (Public 19 · Owner 54 · Admin 84), repartidas en **33 controladores** del backend Rails.

Excluido (solo web): metricas, turnos/shifts, modulo financiero, gestion de contenido y campanas de email, administracion de staff, movimientos de stock, categorias de servicio y configuracion. Esos endpoints existen en el backend pero la app movil no los usa.

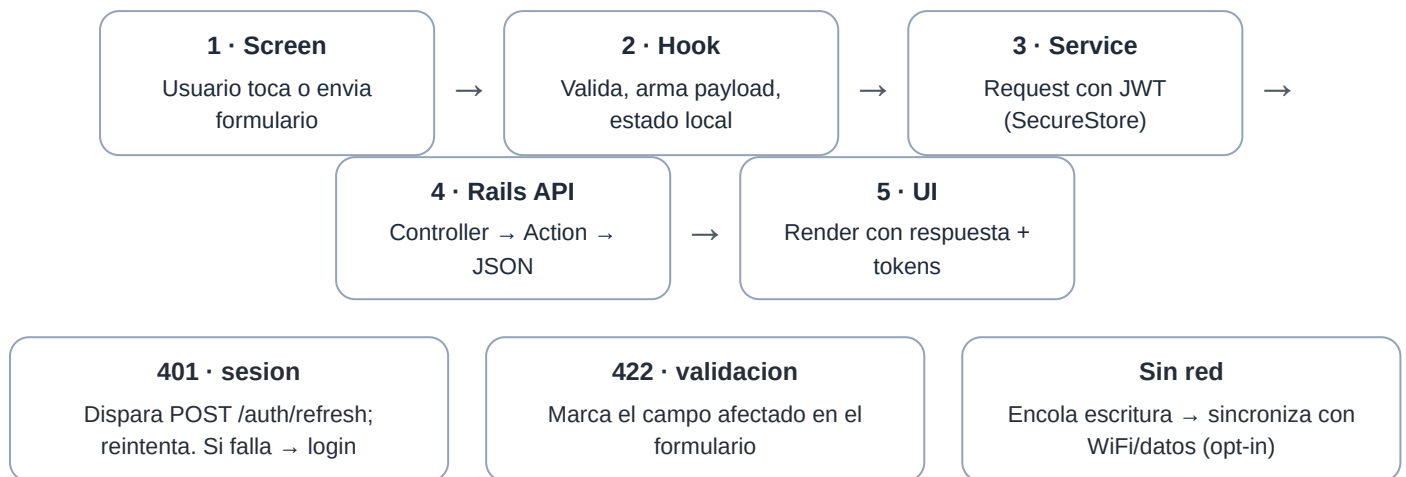
1. Diagrama de capas

Flujo vertical cliente → backend. La app movil es el cliente; el backend Rails se muestra solo como frontera de integracion.



2. Diagrama de flujo de una peticion

Ciclo de vida de una accion del usuario (ej. agendar cita, crear mascota).



Manejo de errores y operacion offline (RNF-REL / RNF-MOB-002).

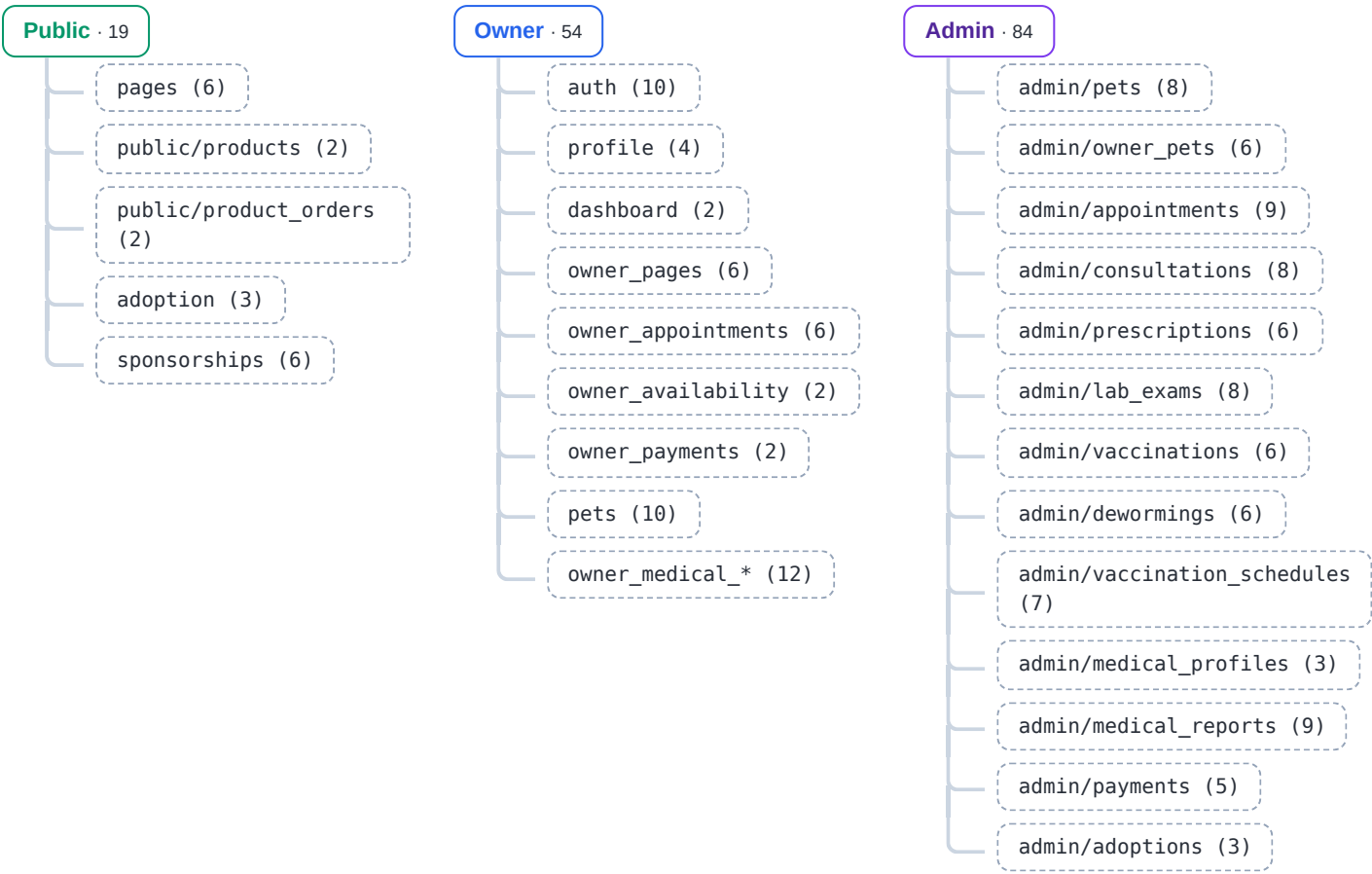
3. Diagrama de navegacion

RootNavigator decide el arbol segun el estado de sesion y el rol. Columna izquierda: acceso publico / pre-login. Columna derecha: navegadores autenticados (owner y staff).



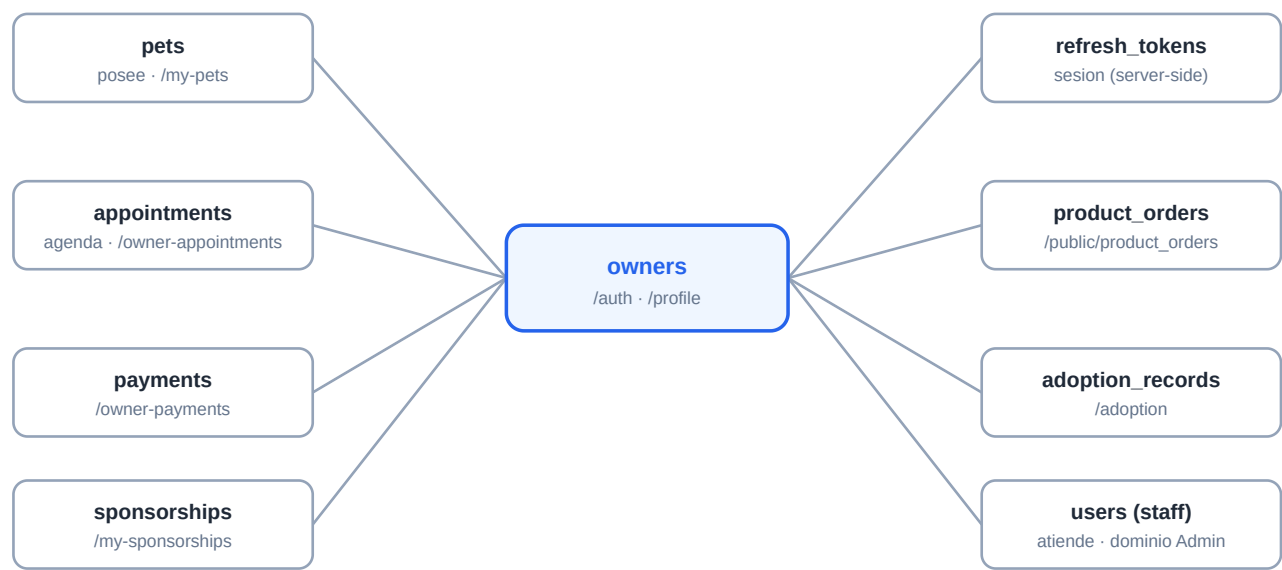
4. Diagrama de dominios y grupos de rutas

Las 157 rutas móviles agrupadas por dominio y controlador (numero de rutas entre parentesis).

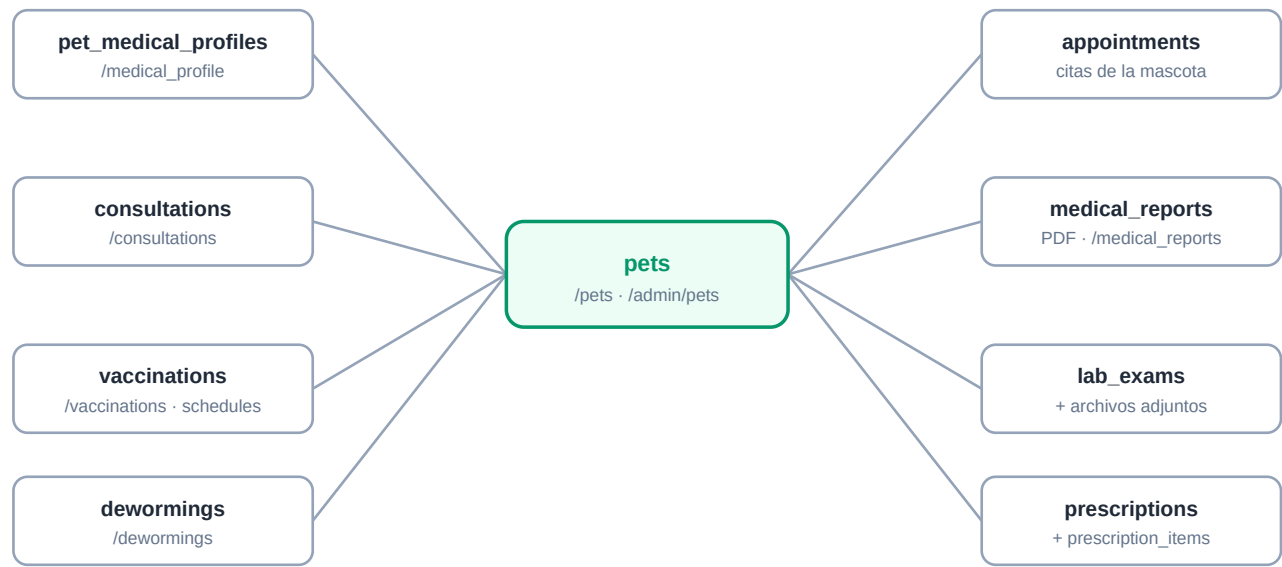


5. Diagrama de entidades (datos que toca la app movil)

Solo el subconjunto de tablas accesibles desde rutas moviles. Diagramas hub centrados en las dos entidades principales: Owner y Pet.



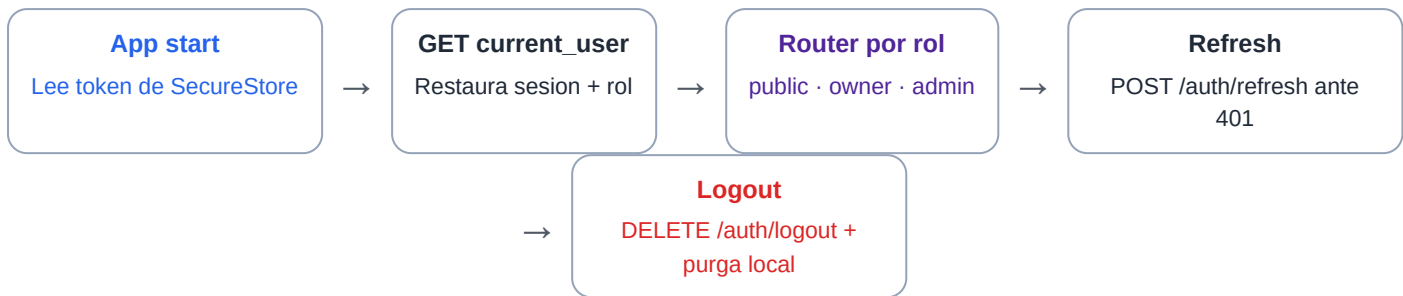
Hub 1 — **owners** y sus relaciones directas.



Hub 2 — **pets**: entidad clinica central. consultations agrupa recetas, examenes y tratamientos.

6. Diagrama de secuencia — sesion y seguridad

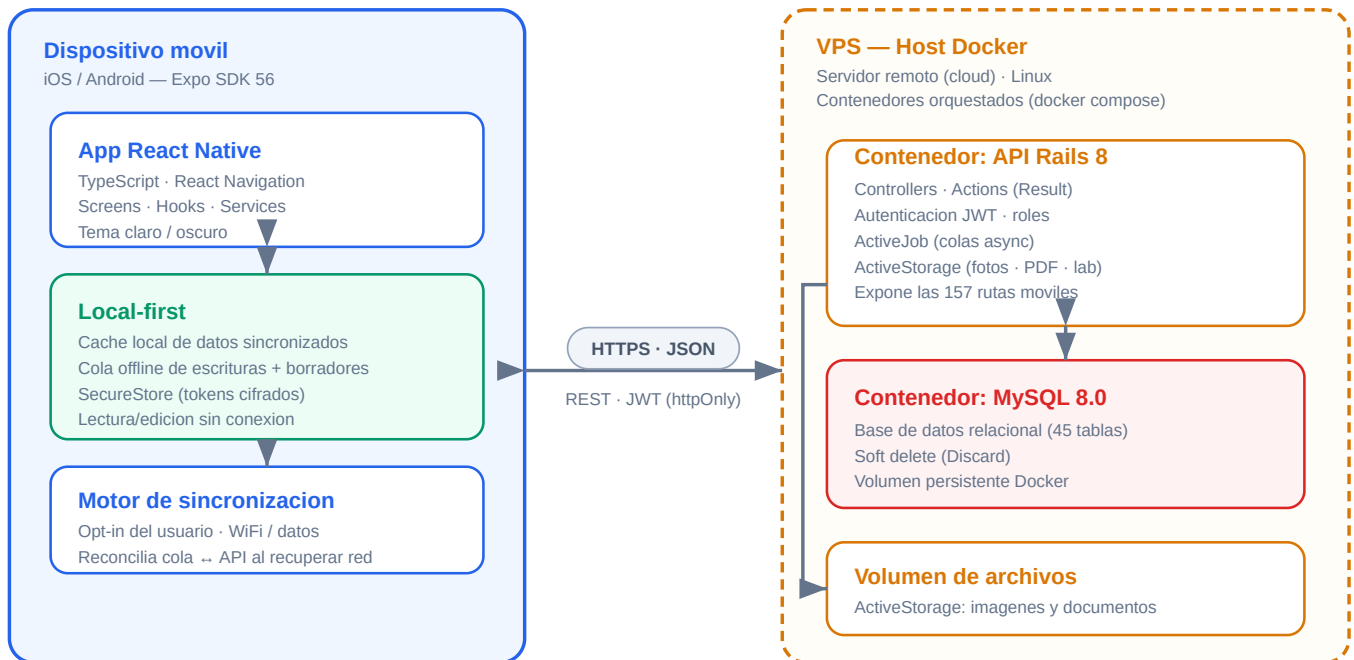
Manejo de tokens (RNF-SEC-001) y control de acceso por rol (RNF-SEC-002).



Un dueño nunca accede a pantallas ni endpoints /admin/*; intentos no autorizados → 403 / redireccion a login.

7. Diagrama de elementos del sistema (despliegue)

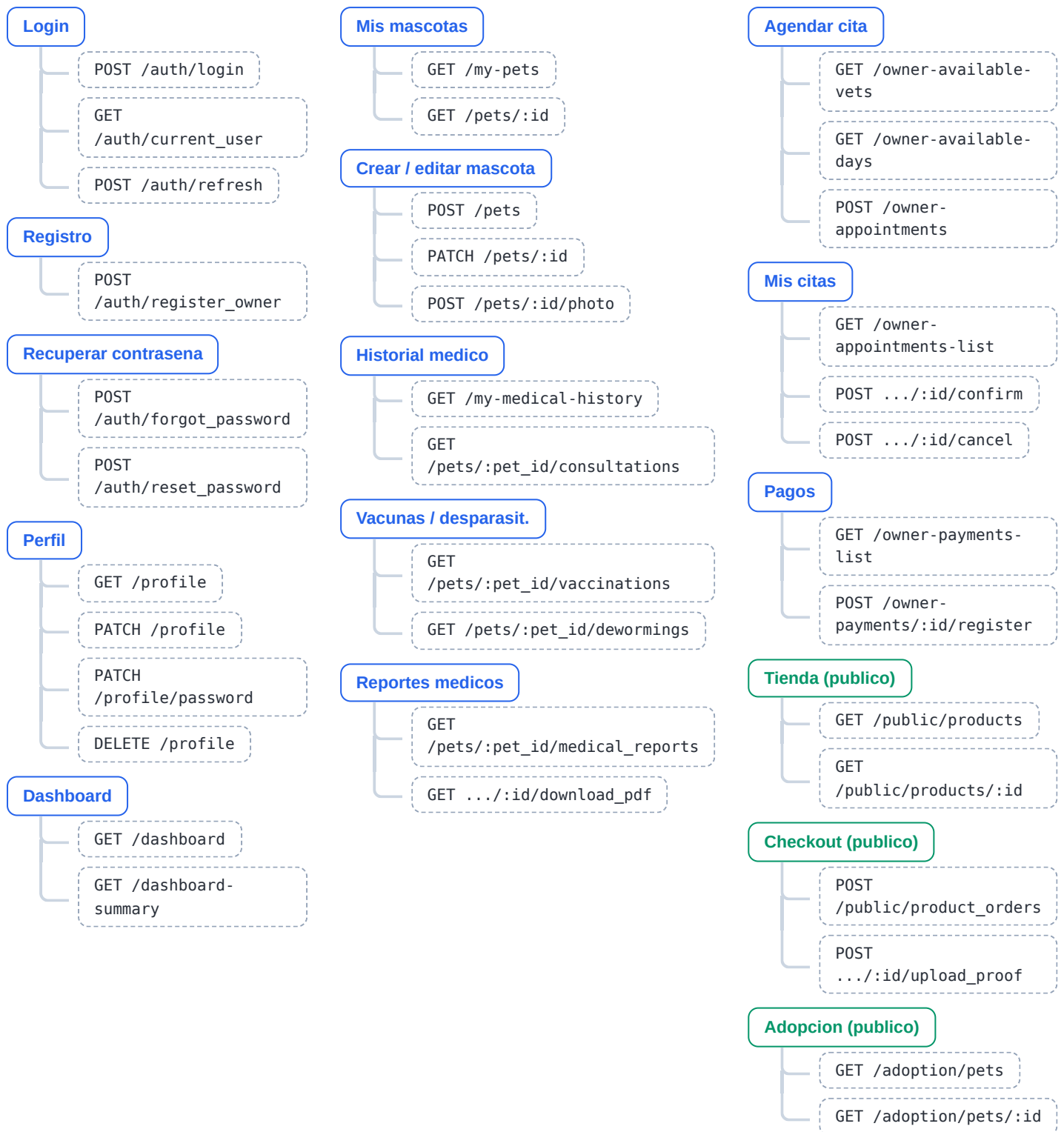
Nodos fisicos/logicos y su comunicacion. El dispositivo movil opera **local-first**; el backend Rails y MySQL corren en contenedores Docker.



El movil es **local-first**: funciona con su copia local y encola cambios; sincroniza con el backend por HTTPS solo con consentimiento del usuario (RNF-MOB-002 / RNF-REL-003). El backend Rails y MySQL se despliegan como contenedores Docker independientes sobre un **VPS** (servidor virtual remoto).

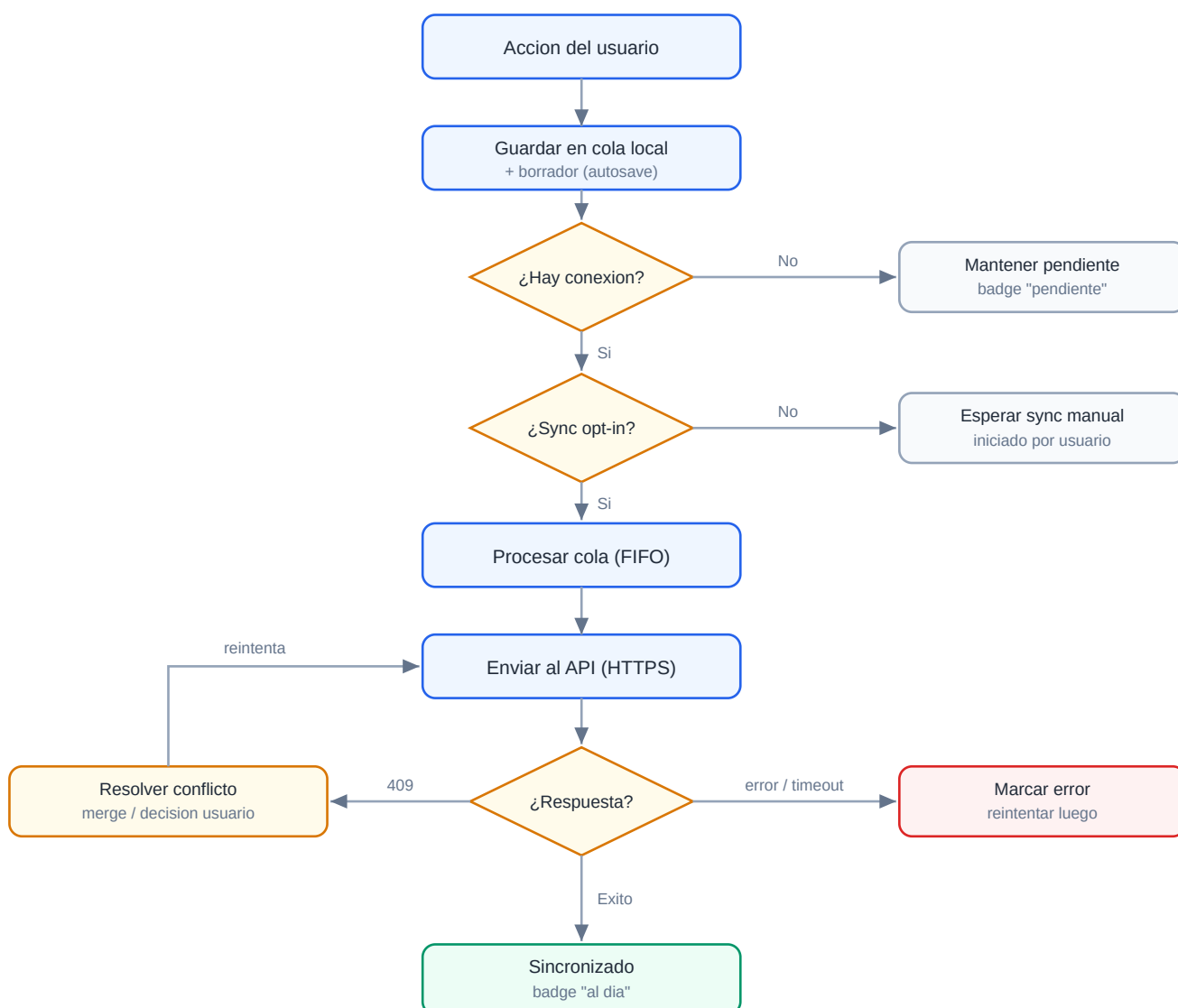
8. Mapa pantalla → endpoints

Cada pantalla principal con las rutas API que dispara. Verde = publico (sin auth) · azul = owner autenticado.



9. Diagrama de sincronizacion (local-first)

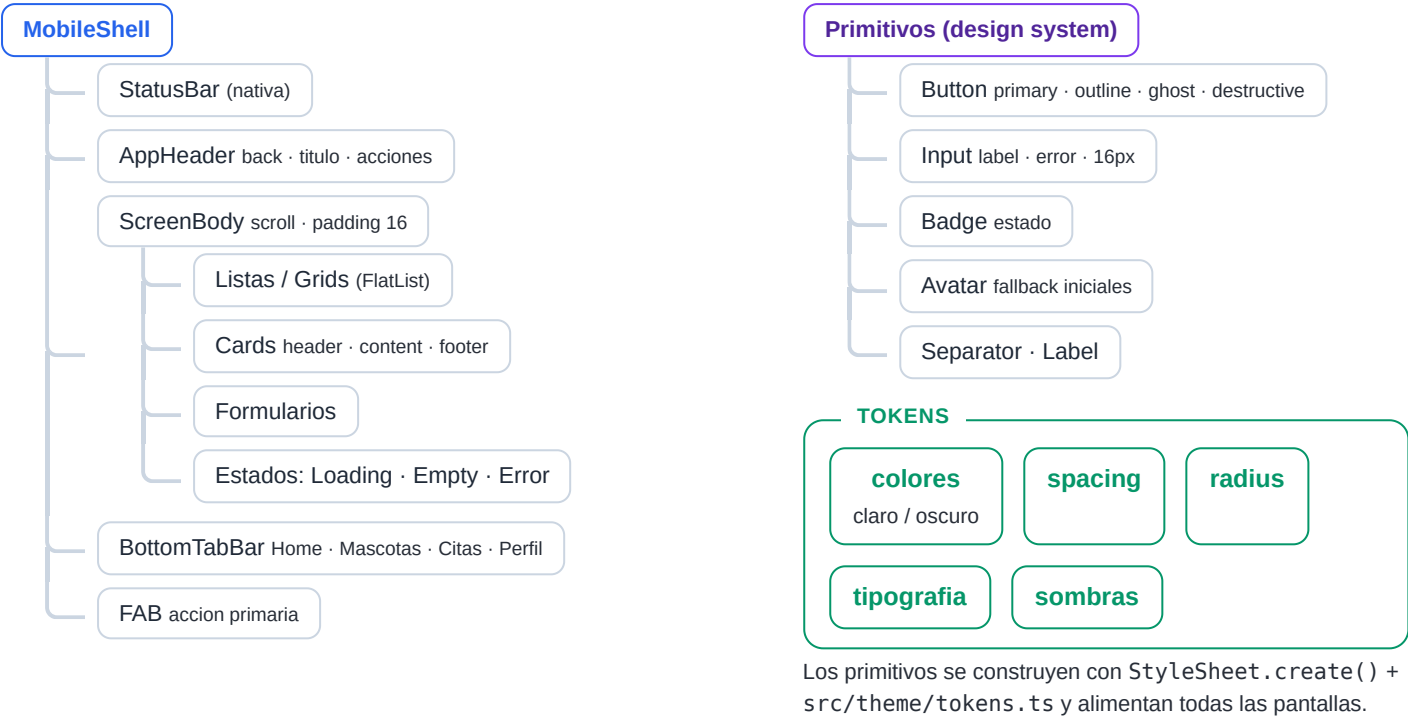
Flujo de la cola offline y el motor de sincronizacion (RNF-MOB-002, RNF-REL-002/003). Rombos = decision.



Sin conexion o sin opt-in, la app nunca simula exito: las escrituras quedan visibles como **pendientes** hasta confirmacion del servidor (RNF-REL-003).

10. Diagrama de componentes UI

Jerarquia de composicion (de mobile-ui-components.md). Los primitivos consumen los tokens del design system.



PawCare Mobile

Requisitos funcionales y no funcionales — aplicacion movil Android (Expo SDK 56 · React Native · TypeScript)

157 rutas API 26 requisitos funcionales 17 no funcionales Public 19 · Owner 54 · Admin 84

Fuente: mobile-requirements.md

Convenciones. **RF-** (funcionales): Descripcion · Prioridad · Justificacion · Criterios de aceptacion. **RNF-** (no funcionales): RNF-{Categoria}-{NNN} + enunciado directo. Prioridades: **Alta** (MVP / legal) · **Media** (operacion completa) · **Baja** (mejora o alcance secundario).

Parte 1 — Requisitos funcionales

PUBLIC — SIN AUTENTICACION (RF-PUB)

RF-PUB-01 Informacion de servicios clinicos Media

La app debe mostrar el catalogo de servicios veterinarios ofrecidos por la clinica, consumiendo la informacion publica del backend, sin requerir autenticacion.

Justificacion. Permite al usuario descubrir la oferta antes de registrarse o agendar una cita. Rutas: GET /services , GET /pages/services .

CRITERIOS DE ACEPTACION

- El usuario puede abrir la pantalla de servicios sin iniciar sesion.
- Se listan los servicios de GET /services con nombre, descripcion y precio cuando el API los provea.
- Ante error de red o respuesta vacia se muestra estado vacio o mensaje comprensible.
- Existe acceso claro a registro, login o agendamiento de cita desde servicios.

RF-PUB-02 Tienda de productos y checkout Media

La app debe permitir consultar productos, ver detalle, crear un pedido y subir comprobante de pago, como flujo publico de compra.

Justificacion. Habilita venta de insumos desde el movil. Rutas: GET /pages/products , GET /public/products , GET /public/products/:id , GET /pages/checkout , POST /public/product_orders , POST /public/product_orders/:id/upload_proof .

CRITERIOS DE ACEPTACION

- Listar productos via GET /public/products y ver detalle via GET /public/products/:id .
- El checkout muestra resumen del pedido antes de confirmar.
- Al confirmar, envia POST /public/product_orders con los datos requeridos.
- Tras crear el pedido, adjuntar comprobante con POST /public/product_orders/:id/upload_proof .
- Mensajes de exito o error segun la respuesta; stock/indisponibilidad reflejado en UI.

RF-PUB-03 Adopcion de mascotas

Media

La app debe exponer el programa de adopcion: landing informativo, listado de mascotas disponibles y ficha de detalle.

Justificacion. Canal publico para promover adopciones responsables. Rutas: GET /adoption , GET /adoption/pets , GET /adoption/pets/:id .

CRITERIOS DE ACEPTACION

- Landing (GET /adoption) con informacion general y enlace al listado.
- Listado (GET /adoption/pets) con datos basicos (nombre, especie, edad, estado).
- Detalle (GET /adoption/pets/:id) con ficha completa, requisitos y accion de contacto/solicitud.
- Solo se muestran mascotas retornadas por el backend; sin datos hardcoded.

RF-PUB-04 Patrocinios (acceso publico)

Baja

La app debe permitir consultar campanas de patrocinio, ver detalle, gastos asociados y crear o actualizar un patrocinio desde el dominio publico.

Justificacion. Canal de donaciones y transparencia de gastos. Rutas: GET /sponsorships , GET /sponsorships/:id , GET /sponsorships/:id/expenses/:expense_id , POST /sponsorships , PATCH/PUT /sponsorships/:id .

CRITERIOS DE ACEPTACION

- Listar patrocinios activos via GET /sponsorships ; detalle con meta, progreso e historial.
- Abrir un gasto via GET /sponsorships/:id/expenses/:expense_id .
- Alta via POST /sponsorships con validacion de campos obligatorios.
- Edicion via PATCH / PUT a /sponsorships/:id reflejada en pantalla.

RF-PUB-05 Contacto y contenido legal

Alta

La app debe mostrar paginas de contacto, terminos de uso y politica de privacidad como contenido informativo publico.

Justificacion. Requisito legal y de confianza para usuarios y tiendas de aplicaciones. Rutas: GET /pages/contact , GET /pages/terms , GET /pages/privacy .

CRITERIOS DE ACEPTACION

- Las tres pantallas accesibles sin autenticacion; contenido desde rutas pages#* .
- Contacto muestra medios oficiales (telefono, email, direccion) cuando el API los provea.
- Terminos y privacidad legibles en movil (scroll, tipografia adecuada).
- Enlaces a terminos y privacidad disponibles desde registro y checkout.

OWNER — DUENO DE MASCOTA (RF-OWN)

RF-OWN-01 Autenticacion y gestion de sesion

Alta

La app debe permitir login, logout, consulta del usuario autenticado y renovacion silenciosa de sesion mediante tokens.

Justificacion. Base de todo el dominio Owner y Admin. Rutas: GET /auth , POST /auth/login , DELETE /auth/logout , GET /auth/current_user , POST /auth/refresh .

CRITERIOS DE ACEPTACION

- Pantalla de bienvenida (GET /auth) con acceso a login y registro.
- Login envia credenciales a POST /auth/login y persiste el token de forma segura.
- Tras login, navega al dashboard/pantalla principal del rol correspondiente.
- GET /auth/current_user al iniciar para restaurar sesion valida.
- POST /auth/refresh antes de expirar o ante 401 recuperable, sin interrumpir al usuario.
- Logout (DELETE /auth/logout) invalida sesion local y remota.
- Credenciales invalidas muestran error sin revelar si el email existe.

RF-OWN-02 Registro de dueno y recuperacion de contrasena

Alta

La app debe permitir registrar un dueno de mascota y completar el flujo de olvido y restablecimiento de contrasena.

Justificacion. Onboarding de nuevos usuarios y recuperacion de acceso. Rutas: POST /auth/register_owner , POST /auth/forgot_password , GET /auth/reset_password/:token , POST /auth/reset_password .

CRITERIOS DE ACEPTACION

- Registro via POST /auth/register_owner con validacion client-side de email, password y campos obligatorios.
- Tras registro, el usuario inicia sesion o queda autenticado segun respuesta del API.
- "Olvide mi contrasena" envia email via POST /auth/forgot_password y confirma en UI.
- El deep link con token abre pantalla de nueva contrasena (GET /auth/reset_password/:token).
- Restablecer via POST /auth/reset_password ; maneja token invalido o expirado.
- Passwords cumplen reglas minimas del backend.

RF-OWN-03 Perfil del dueno

Alta

El dueno autenticado debe poder ver, editar su perfil, cambiar contrasena y eliminar su cuenta.

Justificacion. Autogestion de datos personales y cumplimiento de derechos del usuario. Rutas: GET /profile , PATCH /profile , PATCH /profile/password , DELETE /profile .

CRITERIOS DE ACEPTACION

- GET /profile muestra datos actuales; edicion via PATCH /profile .
- Cambio de contrasena via PATCH /profile/password pidiendo actual y nueva.
- Eliminar cuenta requiere confirmacion explicita antes de DELETE /profile .
- Tras eliminar, la sesion se cierra y no quedan tokens locales.

RF-OWN-04 Dashboard del dueno

Alta

La app debe presentar un resumen del estado del dueno: citas proximas, pagos, alertas medicas y accesos rapidos.

Justificacion. Punto de entrada principal post-login. Rutas: GET /dashboard , GET /dashboard-summary .

CRITERIOS DE ACEPTACION

- Consume GET /dashboard y/o GET /dashboard-summary .
- Muestra al menos: proxima cita, cantidad de mascotas y pagos pendientes cuando el API los incluya.
- Accesos rapidos a mascotas, citas, historial medico y pagos.
- Solo accesible con sesion de dueno valida; ante fallo, estado de error con reintento.

RF-OWN-05 Gestion de mascotas (dueno)

Alta

El dueno debe poder listar, crear, editar, eliminar sus mascotas y gestionar la foto de perfil de cada una.

Justificacion. Entidad central del ecosistema PawCare. Rutas: GET /my-pets , GET /pets , GET /pets/:id , GET /pets/new , GET /pets/:id/edit , POST /pets , PATCH/PUT /pets/:id , DELETE /pets/:id , POST /pets/:id/photo , DELETE /pets/:id/photo .

CRITERIOS DE ACEPTACION

- Listado via GET /my-pets o GET /pets ; creacion via POST /pets con campos validados.
- Edicion via PATCH / PUT ; eliminacion via DELETE con confirmacion.
- Foto via POST / DELETE /pets/:id/photo desde camara o galeria.
- El dueno solo ve/modifica sus mascotas (403/404 manejados); desde detalle accede a historial y citas.

RF-OWN-06 Citas veterinarias (dueno)

Alta

El dueno debe poder listar citas, ver detalle, agendar, editar, confirmar y cancelar citas usando disponibilidad de veterinarios y dias.

Justificacion. Funcionalidad core de la clinica movil. Rutas: GET /my-appointments , GET /owner-appointments-list , GET /owner-appointments/:id , POST /owner-appointments , PATCH /owner-appointments/:id , POST .../confirm , POST .../cancel , GET /owner-available-vets , GET /owner-available-days .

CRITERIOS DE ACEPTACION

- Listado via GET /owner-appointments-list o hub GET /my-appointments ; detalle con estado, mascota, vet y fecha/hora.
- Agendar via POST /owner-appointments tras seleccionar mascota, vet y dia/hora (endpoints de disponibilidad).
- Editar via PATCH ; confirmar via .../confirm ; cancelar via .../cancel con confirmacion.
- Citas pasadas y proximas distinguibles; no se permiten slots no disponibles.

RF-OWN-07 Pagos del dueno

Alta

El dueno debe consultar sus pagos pendientes o realizados y registrar comprobante de pago.

Justificacion. Cierra el ciclo de servicios facturables. Rutas: GET /my-payments , GET /owner-payments-list , POST /owner-payments/:id/register .

CRITERIOS DE ACEPTACION

- Listado via GET /owner-payments-list o hub GET /my-payments .
- Cada pago muestra monto, concepto, estado y fecha de vencimiento si aplica.
- Registrar pago via POST /owner-payments/:id/register con datos y comprobante.
- Tras registro, el estado se actualiza en listado y detalle; pagos ajenos no accesibles.

RF-OWN-08 Historial medico, consultas y resumen

Alta

El dueno debe consultar el historial medico global y por mascota, ver consultas, exportar recetas y marcar tratamientos completados.

Justificacion. Transparencia clinica y continuidad del cuidado. Rutas: GET /my-medical-history , GET /my-pets/:pet_id/medical-history , GET /owner-medical-summary , GET /pets/:pet_id/consultations , GET /owner/consultations/:id/export_recipe , POST /owner/consultations/:id/complete_treatment .

CRITERIOS DE ACEPTACION

- Hub via GET /my-medical-history y resumen via GET /owner-medical-summary .
- Historial por mascota y listado de consultas via las rutas correspondientes.
- Exportar receta via export_recipe y abrir/compartir el archivo.
- Completar tratamiento via complete_treatment con confirmacion; solo registros del dueno.

RF-OWN-09 Perfil medico, vacunas y desparasitaciones

Media

El dueño debe consultar el perfil medico base de su mascota, el historial de vacunas y desparasitaciones.

Justificacion. Informacion preventiva de salud animal. Rutas: GET /pets/:pet_id/medical_profile , GET /pets/:pet_id/vaccinations , GET /pets/:pet_id/dewormings .

CRITERIOS DE ACEPTACION

- Perfil medico via `medical_profile` (alergias, peso, notas segun API).
- Vacunas y desparasitaciones listadas con fechas y tipo.
- La UI indica proximas dosis o vencimientos cuando el backend los calcule.

RF-OWN-10 Reportes medicos y exámenes de laboratorio (dueño)

Media

El dueño debe consultar reportes medicos en PDF, subir archivos de laboratorio y consultar resultados cuando el flujo lo permita.

Justificacion. Acceso documental y participacion en estudios clinicos. Rutas: GET /pets/:pet_id/medical_reports , GET .../:id , GET .../:id/download_pdf , POST /owner/lab_exams/:id/files , PATCH /owner/lab_exams/:id/results .

CRITERIOS DE ACEPTACION

- Listado y detalle de reportes; descarga PDF con apertura en visor o app externa.
- Subida de archivos de lab via `files` ; actualizacion de resultados via `results` cuando corresponda al rol.
- Adjuntos muestran progreso de carga y errores de tamaño o formato.

RF-OWN-11 Patrocinios del dueño

Baja

El dueño autenticado debe ver el listado de sus patrocinios activos desde su area personal.

Justificacion. Continuidad del flujo de donaciones en sesion autenticada. Ruta: GET /my-sponsorships .

CRITERIOS DE ACEPTACION

- Pantalla accesible desde dashboard o menu del dueño.
- Consume GET /my-sponsorships y lista solo patrocinios del usuario.
- Desde cada item se navega al detalle reutilizando RF-PUB-04.

ADMIN — PANEL ADMINISTRATIVO (RF-ADM)

RF-ADM-01 Gestion de mascotas (administrador)

Alta

El staff autorizado debe listar, crear, editar, eliminar mascotas clinicas y gestionar sus fotos desde el panel admin movil.

Justificacion. Operacion diaria de la clinica. Rutas: GET /admin/pets-list , GET /admin/pets/:id , POST /admin/pets , PATCH/PUT /admin/pets/:id , DELETE /admin/pets/:id , POST/DELETE /admin/pets/:id/photo .

CRITERIOS DE ACEPTACION

- Listado con busqueda via GET /admin/pets-list ; CRUD completo segun permisos del rol.
- Gestion de foto via POST/DELETE en /admin/pets/:id/photo .
- Solo usuarios admin autenticados acceden; acciones destructivas con confirmacion.

RF-ADM-02 Gestion de adopciones (administrador)

Media

El staff debe listar solicitudes o registros de adopcion, ver detalle y crear nuevos registros de adopcion.

Justificacion. Gestion interna del programa de adopcion. Rutas: GET /admin/adoptions-list , GET /admin/adoptions/:id , POST /admin/adoptions .

CRITERIOS DE ACEPTACION

- Listado y detalle via las rutas correspondientes; alta via POST con validacion.
- Estados de adopcion mostrados segun respuesta del API.

RF-ADM-03 Consultas clinicas (administrador)

Alta

El staff debe gestionar consultas: listar, crear, editar, eliminar, exportar receta y completar tratamientos.

Justificacion. Nucleo del trabajo veterinario en la app. Rutas: /admin/consultations* (list, show, create, update, destroy), .../export_recipe , .../complete_treatment .

CRITERIOS DE ACEPTACION

- Listado filtrable; formulario envia POST o PATCH segun creacion o edicion.
- Detalle con notas clinicas, mascota, veterinario y estado.
- Exportar receta y completar tratamiento via sus endpoints; eliminar con confirmacion.

RF-ADM-04 Recetas y exámenes de laboratorio (administrador)

Alta

Dentro de una consulta, el staff debe gestionar prescripciones y exámenes de laboratorio, incluyendo archivos adjuntos.

Justificacion. Documentacion clinica completa en consulta. Rutas prescriptions y lab_exams bajo /admin/consultations/:consultation_id/ .

CRITERIOS DE ACEPTACION

- CRUD de prescripciones y de exámenes via los endpoints anidados.
- Subir archivos via .../lab_exams/:id/files ; eliminar via .../files/:file_id .
- Cambios en receta/lab reflejados al volver al detalle de consulta.

RF-ADM-05 Vacunas, desparasitaciones y esquemas

Alta

El staff debe administrar registros de vacunacion, desparasitacion y esquemas programados, incluyendo marcar items completados.

Justificacion. Control preventivo de salud animal. Rutas: /admin/vaccinations* , /admin/dewormings* , /admin/vaccination_schedules* , POST .../complete_item .

CRITERIOS DE ACEPTACION

- CRUD de vacunas, desparasitaciones y esquemas via sus endpoints y listados -list .
- Completar item de esquema via complete_item .
- Listados muestran mascota asociada y fechas relevantes.

RF-ADM-06 Agenda y citas (administrador)

Alta

El staff debe gestionar la agenda: listar citas, crear y editar citas usando disponibilidad de veterinarios, dias y franjas horarias.

Justificacion. Coordinacion operativa de la clinica. Rutas: /admin/appointments* (list, show, available-days, available-vets, time-slots, create, update, destroy).

CRITERIOS DE ACEPTACION

- Listado/calendario via GET /admin/appointments-list .
- Creacion usa vets, dias y slots de disponibilidad antes de POST .
- Editar/cancelar via PATCH/PUT/DELETE; detalle con dueño, mascota, servicio y sala si aplica.
- Conflictos de horario retornados por el API se muestran al usuario.

RF-ADM-07 Pagos (administrador)

Alta

El staff debe listar pagos, ver detalle, registrar pagos recibidos y actualizar su estado.

Justificacion. Control financiero de servicios. Rutas: /admin/payments-list , /admin/payments/:id , POST .../register , PATCH/PUT .../:id .

CRITERIOS DE ACEPTACION

- Listado con filtros por estado si el API los soporta; detalle via GET .
- Registro de pago via register ; actualizacion via PATCH/PUT.
- Comprobantes adjuntos visibles cuando el backend los incluya.

RF-ADM-08 Perfil medico de mascotas (administrador)

Media

El staff debe consultar y editar el perfil medico base de una mascota bajo administracion.

Justificacion. Datos clinicos de referencia para consultas. Rutas: GET/PATCH/PUT /admin/pets/:pet_id/medical_profile .

CRITERIOS DE ACEPTACION

- Visualizacion via GET; edicion via PATCH/PUT con validacion de campos clinicos.
- Cambios persisten y se reflejan en consultas posteriores.

RF-ADM-09 Mascotas de dueños (administrador)

Media

El staff debe poder crear y editar mascotas asociadas a un dueño específico, incluyendo foto.

Justificacion. Alta asistida en recepcion o consulta. Rutas: /admin/owners/:owner_id/pets* (show, create, update, photo).

CRITERIOS DE ACEPTACION

- Creacion via POST /admin/owners/:owner_id/pets ; edicion via PATCH/PUT anidada con owner_id.
- Gestion de foto via endpoints de photo; detalle via GET con datos del dueño y la mascota.

El staff debe listar reportes medicos, generarlos, importarlos, exportarlos en multiples formatos y enviarlos por email.

Justificacion. Documentacion formal y comunicacion con duenos. Rutas: `/admin/medical_reports` (index, show, create, generate_pdf, import, export_csv/json/pdf, email).

CRITERIOS DE ACEPTACION

- Listado y detalle; generar PDF via `generate_pdf` ; importar via `import` .
- Exportar CSV, JSON y PDF via endpoints `export_*` ; enviar por email via `email` .
- Archivos generados se pueden abrir o compartir desde el dispositivo.

Parte 2 — Requisitos no funcionales

Formato RNF-{Categoria}-{NNN}: Titulo . Categorias: ARQ (arquitectura), MOB (movil), SEC (seguridad), UX (experiencia), REL (confiabilidad), PERF (rendimiento), ARC (archivos), PRI (privacidad), OBS (observabilidad), CMP (compatibilidad), MAINT (mantenibilidad).

RNF-ARQ-001 Plataforma movil Android

ARQ

La aplicacion debe desarrollarse con **Expo SDK 56**, **React Native** y **TypeScript**, como cliente nativo **solo para Android**. El codigo de pantallas y servicios debe estar tipado de forma estricta. Los flujos principales no deben depender de contenedores web embebidos. La version de Expo debe mantenerse alineada con la documentacion oficial v56 (`AGENTS.md` , `docs/arq.md`). No se incluyen targets, builds ni dependencias especificas de iOS.

RNF-ARQ-002 Integracion con API Rails

ARQ

La app debe consumir exclusivamente las **157 rutas** documentadas en `mobile-routes.md` , sin reimplementar logica de negocio en el cliente. Debe existir un cliente HTTP centralizado con base URL configurable por entorno (desarrollo, staging, produccion). Los payloads y respuestas deben estar tipados segun los contratos del API Rails existente.

RNF-MOB-001 Mobile First

MOB

La aplicacion debe disenarse y construirse con enfoque **mobile first**: layouts, navegacion, tipografia y patrones de interaccion pensados primero para pantallas de telefono (una mano, orientacion vertical, safe areas). Las adaptaciones a tablet, si existen en el futuro, deben ser extensiones del diseno movil base y no condicionar la arquitectura. Toda pantalla de `mobile-screens.md` debe priorizar legibilidad y acciones principales en viewports desde **320 px** de ancho logico.

RNF-MOB-002 Auto-sincronizacion bajo consentimiento del usuario

MOB

La aplicacion debe ser capaz de **auto-sincronizarse** con el backend cuando detecte conexion activa por **WiFi o datos moviles**, **si y solo si el usuario lo ha autorizado** previamente en ajustes (opt-in explicito, revocable en cualquier momento).

COMPORTAMIENTO ESPERADO

- Sin autorizacion, **ninguna** sincronizacion automatica en segundo plano; solo sincronizacion manual.
- Con autorizacion, al detectar conectividad procesa la cola local (formularios, uploads, acciones diferidas) y refresca datos stale segun prioridad.
- El usuario elige, como minimo, sincronizar solo con **WiFi** o tambien con **datos moviles**.
- Indicador discreto de estado (sincronizando / al dia / pendiente / error) sin interrumpir salvo conflicto que requiera decision.

RNF-SEC-001 Seguridad de sesion y almacenamiento de tokens

SEC

Los tokens de autenticacion y demas secretos deben almacenarse en el almacenamiento seguro del sistema operativo cuando este disponible, **nunca** en almacenamiento plano sin cifrar. Toda comunicacion con el backend por **HTTPS**. El refresh via `POST /auth/refresh` sin exponer tokens en logs de produccion. Al cerrar sesion (`DELETE /auth/logout`) deben eliminarse todos los secretos y datos de sesion locales.

RNF-SEC-002 Control de acceso por rol y dominio

SEC

La navegacion y las llamadas API deben restringirse segun rol: **publico**, **dueno (owner)** o **administrador (admin)**. Un dueno autenticado no debe acceder a pantallas ni endpoints `/admin/*`. El staff admin solo accede al dominio Admin segun permisos de `GET /auth/current_user`. Los intentos no autorizados redirigen a login o informan error **403** de forma clara.

RNF-UX-001 Experiencia de usuario movil

UX

La interfaz debe cumplir patrones moviles de produccion en Android: areas tactiles minimas de **48×48 dp** (Material), inputs con **font-size ≥ 16 px**, estados de carga / vacio / error en todo listado o formulario, navegacion por tabs y stacks segun `mobile-screens.md`, y respeto de **safe areas** y barras del sistema (status bar, navigation bar, recortes de pantalla). Los mensajes deben estar en espanol claro, sin stack traces ni jerga tecnica.

RNF-UX-002 Design system y tema claro/oscuro

UX

La UI debe implementar los tokens centralizados de `mobile-design-tokens.css` y `src/theme/tokens.ts`, incluyendo **modo claro y modo oscuro** coherentes con `mobile-ui-components.md`. El tema puede seguir la preferencia del sistema o una seleccion manual del usuario.

RNF-REL-001 Tolerancia a fallos

REL

El sistema debe manejar los errores comunes con elegancia — interrupciones de red, timeouts, respuestas **5xx** o token expirado — e informar al usuario, intentando **preservar los datos ingresados** (borrador local, cola de reintentos, no descartar formularios). Los errores **422** deben senalar el campo afectado. Los **401** deben disparar refresh de sesion o re-autenticacion segun corresponda.

RNF-REL-002 Copia de seguridad local y recuperacion

REL

RPO: ante cierre inesperado o fallo del dispositivo, la perdida de datos no sincronizados encolados no debe exceder el **ultimo guardado automatico de borrador** (intervalo maximo recomendado: **5 minutos** mientras se edita, o al perder foco del formulario).

RTO: tras reiniciar la app, el usuario debe poder retomar operaciones pendientes o borradores recuperables en un plazo maximo de **30 segundos** desde el arranque, sin reingresar datos ya capturados localmente.

Los datos clinicos descargados no deben depender de cache permanente sin cifrar; la recuperacion aplica a **cola de sincronizacion, borradores y estado de sesion** autorizado.

RNF-REL-003 Operacion offline controlada

REL

Sin conexion, la app debe permitir consultar la **ultima copia local** de datos ya sincronizados (cuando exista) y encolar acciones de escritura para envio posterior. Las acciones encoladas deben mostrarse como **pendientes de sincronizacion**. No debe simular exito de operaciones no confirmadas por el servidor.

RNF-PERF-001 Rendimiento en red movil

PERF

Las transiciones entre pantallas principales deben percibirse en **menos de 1 segundo** con conexion normal. Las operaciones largas (upload/download) no deben bloquear la UI; deben mostrar progreso y permitir cancelacion cuando el SO lo permita. Los listados deben soportar paginacion si el API la ofrece.

RNF-ARC-001 Manejo de archivos y multimedia

ARC

La app debe soportar captura y subida de imagenes (fotos de mascota, comprobantes) y descarga o visualizacion de **PDFs medicos** y archivos de laboratorio, conforme a las rutas de upload/download del API. Debe respetar limites de tamano y tipo MIME del backend. Los permisos de camara, galeria y almacenamiento se solicitan en runtime con mensaje contextual.

RNF-PRI-001 Privacidad y datos de salud animal

PRI

Los datos medicos y personales deben tratarse con minima exposicion: sin cache innecesaria en disco, sin logs con informacion identificable en builds de produccion, y con acceso a la politica de privacidad (GET /pages/privacy) desde registro y perfil. La eliminacion de cuenta debe purgar datos locales. La auto-sincronizacion (RNF-MOB-002) nunca debe activarse sin consentimiento explicito.

RNF-OBS-001 Observabilidad y registro de errores

OBS

Los errores no controlados deben registrarse con contexto tecnico en entorno de **desarrollo** unicamente. En produccion, el usuario recibe mensajes accionables; el equipo tecnico correlaciona fallos mediante identificadores de error anonimos, sin incluir tokens, contrasenas ni diagnosticos veterinarios completos.

RNF-CMP-001 Compatibilidad de dispositivos Android

CMP

La app debe ser compatible con las versiones de **Android** soportadas por Expo SDK 56 segun app.json (API level minimo definido en el proyecto). Debe funcionar en telefonos pequenos (320 px de ancho logico) y grandes, con interaccion tactil completa (sin depender de teclado fisico). Debe validarse manualmente en al menos un dispositivo Android fisico y un emulador durante QA.

RNF-MAINT-001 Mantenibilidad y trazabilidad documental

MAINT

El codigo debe mantener trazabilidad entre requisitos (**RF-***, **RNF-***), rutas API (mobile-routes.md), pantallas (mobile-screens.md) y componentes (mobile-ui-components.md). Los servicios API deben agruparse por dominio en src/services/. Cambios en rutas incluidas deben actualizar la documentacion en docs/. El demo HTML es referencia visual, no codigo de produccion.

Matriz de trazabilidad resumida

DOMINIO	REQUISITOS FUNCIONALES	RUTAS API (APROX.)
Public	RF-PUB-01 a RF-PUB-05	19
Owner	RF-OWN-01 a RF-OWN-11	54
Admin	RF-ADM-01 a RF-ADM-10	84
No funcionales	RNF-ARQ, MOB, SEC, UX, REL, PERF, ARC, PRI, OBS, CMP, MAINT	—

Apendice — Cobertura endpoint → RF

Mapeo de las **157 rutas** de `mobile-routes.md` a su requisito funcional. Cobertura: **156 / 157** endpoints. Unico endpoint sin RF: `GET /auth/debug` (diagnostico, fuera del alcance funcional).

Public — 19 rutas

METODO · RUTA	RF
pages	
GET /services	RF-PUB-01
GET /products	RF-PUB-02
GET /checkout	RF-PUB-02
GET /contact	RF-PUB-05
GET /terms	RF-PUB-05
GET /privacy	RF-PUB-05
public/products · public/product_orders	
GET /public/products	RF-PUB-02
GET /public/products/:id	RF-PUB-02
POST /public/product_orders	RF-PUB-02
POST /public/product_orders/:id/upload_proof	RF-PUB-02
adoption	
GET /adoption	RF-PUB-03
GET /adoption/pets	RF-PUB-03
GET /adoption/pets/:id	RF-PUB-03
sponsorships	
GET /sponsorships	RF-PUB-04
GET /sponsorships/:id	RF-PUB-04
GET /sponsorships/:id/expenses/:expense_id	RF-PUB-04
POST /sponsorships	RF-PUB-04
PATCH /sponsorships/:id	RF-PUB-04
PUT /sponsorships/:id	RF-PUB-04

Owner — 54 rutas

METODO · RUTA	RF
auth	
GET /auth	RF-OWN-01
GET /auth/current_user	RF-OWN-01
GET /auth/debug	<i>sin RF (diagnostico)</i>
POST /auth/login	RF-OWN-01
POST /auth/refresh	RF-OWN-01
DELETE /auth/logout	RF-OWN-01
POST /auth/register_owner	RF-OWN-02
POST /auth/forgot_password	RF-OWN-02
GET /auth/reset_password/:token	RF-OWN-02
POST /auth/reset_password	RF-OWN-02
profile	
GET /profile	RF-OWN-03
PATCH /profile	RF-OWN-03
PATCH /profile/password	RF-OWN-03
DELETE /profile	RF-OWN-03
dashboard	
GET /dashboard	RF-OWN-04
GET /dashboard-summary	RF-OWN-04
owner_pages (hubs)	
GET /my-pets	RF-OWN-05
GET /my-appointments	RF-OWN-06
GET /my-payments	RF-OWN-07
GET /my-medical-history	RF-OWN-08
GET /my-pets/:pet_id/medical-history	RF-OWN-08
GET /my-sponsorships	RF-OWN-11
owner_appointments · owner_availability	
GET /owner-appointments-list	RF-OWN-06
GET /owner-appointments/:id	RF-OWN-06
POST /owner-appointments	RF-OWN-06
PATCH /owner-appointments/:id	RF-OWN-06
POST /owner-appointments/:id/confirm	RF-OWN-06
POST /owner-appointments/:id/cancel	RF-OWN-06
GET /owner-available-vets	RF-OWN-06
GET /owner-available-days	RF-OWN-06
owner_payments	
GET /owner-payments-list	RF-OWN-07
POST /owner-payments/:id/register	RF-OWN-07
owner_medical_summary · owner_consultations	
GET /owner-medical-summary	RF-OWN-08
GET /pets/:pet_id/consultations	RF-OWN-08
GET /owner/consultations/:id/export_recipe	RF-OWN-08
POST /owner/consultations/:id/complete_treatment	RF-OWN-08

METODO · RUTA	RF
pets	
GET /pets	RF-OWN-05
GET /pets/new	RF-OWN-05
GET /pets/:id	RF-OWN-05
GET /pets/:id/edit	RF-OWN-05
POST /pets	RF-OWN-05
PATCH /pets/:id	RF-OWN-05
PUT /pets/:id	RF-OWN-05
DELETE /pets/:id	RF-OWN-05
POST /pets/:id/photo	RF-OWN-05
DELETE /pets/:id/photo	RF-OWN-05
owner_medical_profiles · vaccinations · dewormings	
GET /pets/:pet_id/medical_profile	RF-OWN-09
GET /pets/:pet_id/vaccinations	RF-OWN-09
GET /pets/:pet_id/dewormings	RF-OWN-09
owner_lab_exams · owner_medical_reports	
POST /owner/lab_exams/:id/files	RF-OWN-10
PATCH /owner/lab_exams/:id/results	RF-OWN-10
GET /pets/:pet_id/medical_reports	RF-OWN-10
GET /pets/:pet_id/medical_reports/:id	RF-OWN-10
GET /pets/:pet_id/medical_reports/:id/download_pdf	RF-OWN-10

METODO · RUTA	RF
admin/pets	
GET /admin/pets-list	RF-ADM-01
GET /admin/pets/:id	RF-ADM-01
POST /admin/pets	RF-ADM-01
PATCH /admin/pets/:id	RF-ADM-01
PUT /admin/pets/:id	RF-ADM-01
DELETE /admin/pets/:id	RF-ADM-01
POST /admin/pets/:id/photo	RF-ADM-01
DELETE /admin/pets/:id/photo	RF-ADM-01
admin/adoptions	
GET /admin/adoptions-list	RF-ADM-02
GET /admin/adoptions/:id	RF-ADM-02
POST /admin/adoptions	RF-ADM-02
admin/consultations	
GET /admin/consultations-list	RF-ADM-03
GET /admin/consultations/:id	RF-ADM-03
GET /admin/consultations/:id/export_recipe	RF-ADM-03
POST /admin/consultations	RF-ADM-03
POST /admin/consultations/:id/complete_treatment	RF-ADM-03
PATCH /admin/consultations/:id	RF-ADM-03
PUT /admin/consultations/:id	RF-ADM-03
DELETE /admin/consultations/:id	RF-ADM-03
admin/prescriptions (consultations anidadas)	
GET .../prescriptions	RF-ADM-04
GET .../prescriptions/:id	RF-ADM-04
POST .../prescriptions	RF-ADM-04
PATCH .../prescriptions/:id	RF-ADM-04
PUT .../prescriptions/:id	RF-ADM-04
DELETE .../prescriptions/:id	RF-ADM-04
admin/lab_exams (consultations anidadas)	
GET .../lab_exams	RF-ADM-04
GET .../lab_exams/:id	RF-ADM-04
POST .../lab_exams	RF-ADM-04
POST .../lab_exams/:id/files	RF-ADM-04
PATCH .../lab_exams/:id	RF-ADM-04
PUT .../lab_exams/:id	RF-ADM-04
DELETE .../lab_exams/:id	RF-ADM-04
DELETE .../lab_exams/:id/files/:file_id	RF-ADM-04
admin/vaccinations	
GET /admin/vaccinations-list	RF-ADM-05
GET /admin/vaccinations/:id	RF-ADM-05
POST /admin/vaccinations	RF-ADM-05
PATCH /admin/vaccinations/:id	RF-ADM-05

METODO · RUTA	RF
PUT /admin/vaccinations/:id	RF-ADM-05
DELETE /admin/vaccinations/:id	RF-ADM-05
admin/dewormings	
GET /admin/dewormings-list	RF-ADM-05
GET /admin/dewormings/:id	RF-ADM-05
POST /admin/dewormings	RF-ADM-05
PATCH /admin/dewormings/:id	RF-ADM-05
PUT /admin/dewormings/:id	RF-ADM-05
DELETE /admin/dewormings/:id	RF-ADM-05
admin/vaccination_schedules	
GET /admin/vaccination-schedules-list	RF-ADM-05
GET /admin/vaccination_schedules/:id	RF-ADM-05
POST /admin/vaccination_schedules	RF-ADM-05
POST /admin/vaccination_schedules/:id/complete_item	RF-ADM-05
PATCH /admin/vaccination_schedules/:id	RF-ADM-05
PUT /admin/vaccination_schedules/:id	RF-ADM-05
DELETE /admin/vaccination_schedules/:id	RF-ADM-05
admin/appointments	
GET /admin/appointments-list	RF-ADM-06
GET /admin/appointments/:id	RF-ADM-06
GET /admin/appointments/available-days	RF-ADM-06
GET /admin/appointments/available-vets	RF-ADM-06
GET /admin/appointments/time-slots	RF-ADM-06
POST /admin/appointments	RF-ADM-06
PATCH /admin/appointments/:id	RF-ADM-06
PUT /admin/appointments/:id	RF-ADM-06
DELETE /admin/appointments/:id	RF-ADM-06
admin/payments	
GET /admin/payments-list	RF-ADM-07
GET /admin/payments/:id	RF-ADM-07
POST /admin/payments/:id/register	RF-ADM-07
PATCH /admin/payments/:id	RF-ADM-07
PUT /admin/payments/:id	RF-ADM-07
admin/medical_profiles	
GET /admin/pets/:pet_id/medical_profile	RF-ADM-08
PATCH /admin/pets/:pet_id/medical_profile	RF-ADM-08
PUT /admin/pets/:pet_id/medical_profile	RF-ADM-08
admin/owner_pets	
GET /admin/owners/:owner_id/pets/:id	RF-ADM-09
POST /admin/owners/:owner_id/pets	RF-ADM-09
PATCH /admin/owners/:owner_id/pets/:id	RF-ADM-09
PUT /admin/owners/:owner_id/pets/:id	RF-ADM-09
POST /admin/owners/:owner_id/pets/:id/photo	RF-ADM-09

METODO · RUTA	RF
DELETE /admin/owners/:owner_id/pets/:id/photo	RF-ADM-09
admin/medical_reports	
GET /admin/medical_reports	RF-ADM-10
GET /admin/medical_reports/:id	RF-ADM-10
GET /admin/medical_reports/:id/export_csv	RF-ADM-10
GET /admin/medical_reports/:id/export_json	RF-ADM-10
GET /admin/medical_reports/:id/export_pdf	RF-ADM-10
GET /admin/medical_reports/generate_pdf	RF-ADM-10
POST /admin/medical_reports	RF-ADM-10
POST /admin/medical_reports/:id/email	RF-ADM-10
POST /admin/medical_reports/import	RF-ADM-10